

ДЕПОНИРУЕМЫЕ МАТЕРИАЛЫ

Программа для ЭВМ

«Цифровой помощник»

Реферат, описание пользовательского интерфейса и фрагменты исходного текста,
идентифицирующие программу

Параметр	Значение
Версия программы	2.4.6.2 STABLE
Дата версии	24.07.2026
Дата формирования материалов	28.07.2026
Тип программы	Прогрессивное веб-приложение (PWA)
Исходный текст	609465 байт, 17905 строк, 13 файлов HTML/JavaScript/CSS

Сведения об авторе и правообладателе указываются заявителем в заявлении на государственную регистрацию.

1. Реферат

Программа для ЭВМ «Цифровой помощник» представляет собой мобильное прогрессивное веб-приложение для планирования и сопровождения рабочих смен машинистов и помощников машиниста. Программа формирует календарь смен, подбирает нормативы маршрутов, строит последовательность рейсов по номерам поездов, отображает расписание и текущую станцию, рассчитывает рабочее время, пробег и ориентировочную оплату, ведет предупреждения и заметки, хранит QR-код АСПОЖ и экспортирует график в iCalendar. Предусмотрены автономная работа, локальное кеширование справочников и поездных данных, уведомления и обновление PWA. Область применения — информационная поддержка локомотивных бригад. Языки: JavaScript (ECMAScript), HTML5, CSS3. Среда выполнения: современный веб-браузер с поддержкой PWA на Android, iOS, Windows или macOS. Объем исходного текста: 609465 байт.

Объем реферата: 846 знаков с пробелами.

2. Назначение и область применения

«Цифровой помощник» предназначен для персональной информационной поддержки локомотивной бригады при подготовке к рабочей смене и во время ее выполнения. Программа объединяет график смен, нормативные данные маршрута, сведения о поездах, расписание остановок, предупреждения, заметки и месячную статистику.

Основная область применения — оперативная деятельность машинистов и помощников машиниста на железнодорожных направлениях. Предметные справочники текущей версии ориентированы прежде всего на Ярославское направление. Программа является вспомогательным информационным средством и не заменяет обязательные нормативные документы и распоряжения перевозчика.

3. Решаемые задачи

- ведение персонального календаря дневных, ночных и утренних смен;
- поиск норматива смены по маршруту, календарной дате и дню недели;
- построение последовательности рейсов по номерам поездов;
- отображение маршрута, остановок, времени хода, оборотов и пробега;
- сопоставление станций со справочником и отображение предупреждений;
- учет рабочих и ночных часов, расчет месячной статистики и ориентировочной оплаты;
- хранение заметок к сменам и станциям, а также QR-кода АСПОЖ;
- экспорт графика смен в файл iCalendar (.ics);
- автономная работа с локальными справочниками и кешированными данными;
- уведомления по событиям смены и доставка обновлений PWA.

4. Функциональные возможности

Подсистема	Основные функции
Календарь смен	Месячная сетка, конструктор смены, связанные ночь/утро, заметки, экспорт .ics.
Маршрут	До 10 поездов, cache-first поиск ниток, карточки рейсов, пробег, обороты, тормозные точки и предупреждения.
Расписание	Таймлайн остановок, текущая и следующая станция, live-часы, заметки и уведомления.
Личный кабинет	Табельный номер, должность, ближайшая смена, часы, норма, ориентировочная оплата и QR АСПОЖ.
Автономная работа	Service Worker, Cache API, localStorage, локальные JSON-справочники и резерв поездных данных.

5. Архитектура, языки и среда выполнения

Программа реализована как статическое одностраничное приложение. Файл index.html содержит разметку экранов и основную бизнес-логику на JavaScript; девять CSS-файлов разделяют оформление по функциональным областям; sw.js реализует автономную оболочку и уведомления. Сборщик и JavaScript-фреймворк в runtime не используются.

Характеристика	Значение
Языки	JavaScript (ECMAScript), HTML5, CSS3
Web API	Fetch, Service Worker, Cache API, localStorage, Notifications, Web Share, Canvas, FileReader
Среда выполнения	Современный веб-браузер с поддержкой PWA; статический HTTPS-хостинг
Операционные системы	Android, iOS, Windows, macOS и иные ОС с совместимым браузером
Внешние источники	API расписаний через прокси; Google Apps Script для обратной связи; CloudTips и Telegram как внешние переходы
Runtime-зависимости	Отсутствуют; клиент написан на Vanilla JavaScript

6. Краткая история разработки

Git-история содержит 53 коммита одного автора за период 24.06.2026–24.07.2026. Первый доступный коммит a205197 фиксирует версию 2.3.4; более ранние версии 2.1.0–2.3.3 описаны в журнале Version/Commit.txt, но отсутствуют как самостоятельные коммиты.

Дата / коммит	Этап
24.06.2026 · a205197	Стабилизация навигации версии 2.3.4.
27.06.2026 · 6611093	Кеш поездов и локальная резервная база.
07.07.2026 · 240e8cf	Переход к offline-first PWA.
11.07.2026 · aa7a600	Ребрендинг в «Цифровой помощник».
14.07.2026 · 3e45746	Уведомления и локальная база смен.
24.07.2026 · 83b9a82	Экспорт смен в универсальный формат .ics.
24.07.2026 · 1216024	Стабильный релиз 2.4.6.2.

7. Описание пользовательского интерфейса

7.1. Экран входа

Стартовый экран содержит логотип, название программы, поле табельного номера и кнопку входа. До создания локальной сессии остальные экраны заблокированы. Введенный номер используется для разделения персональных данных на устройстве.

7.2. График смен

Главный экран представляет календарь месяца с цветовой маркировкой смен. Пользователь выбирает день и маршрут, просматривает явку, рабочее время и поезда, добавляет заметку или переходит к построению маршрута. График месяца можно экспортировать в .ics.

7.2.1. Визуальное представление экрана графика смен

Месячный обзор показывает распределение дневных, ночных и технических смен. После выбора даты открывается сводка с маршрутом, временем работы, поездами, расчетными показателями и средствами изменения маршрута.

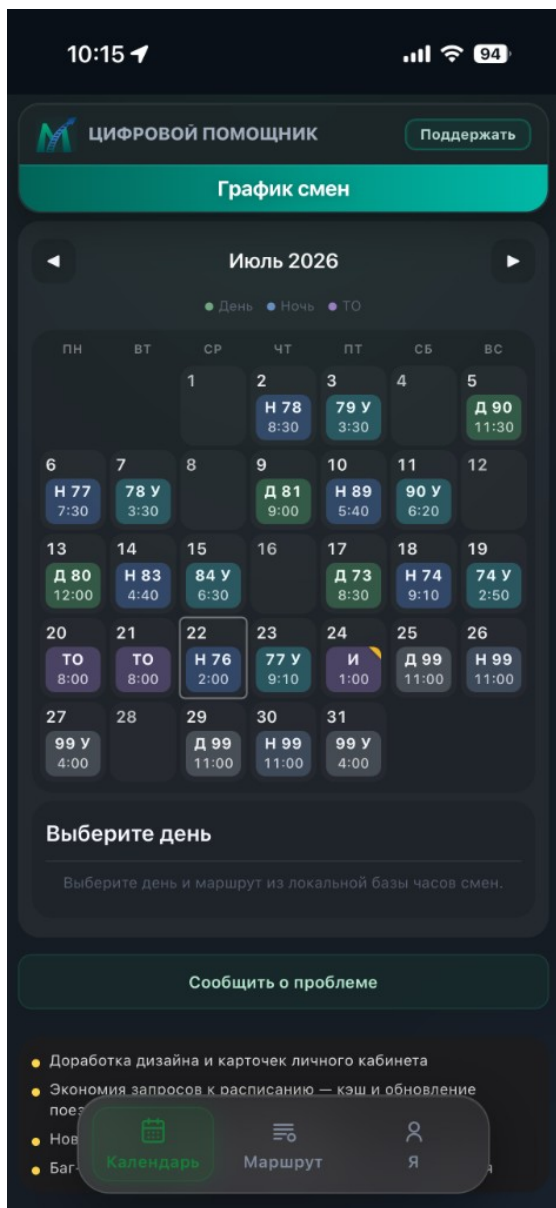


Рисунок 1 — Месячный календарь смен с цветовой маркировкой.

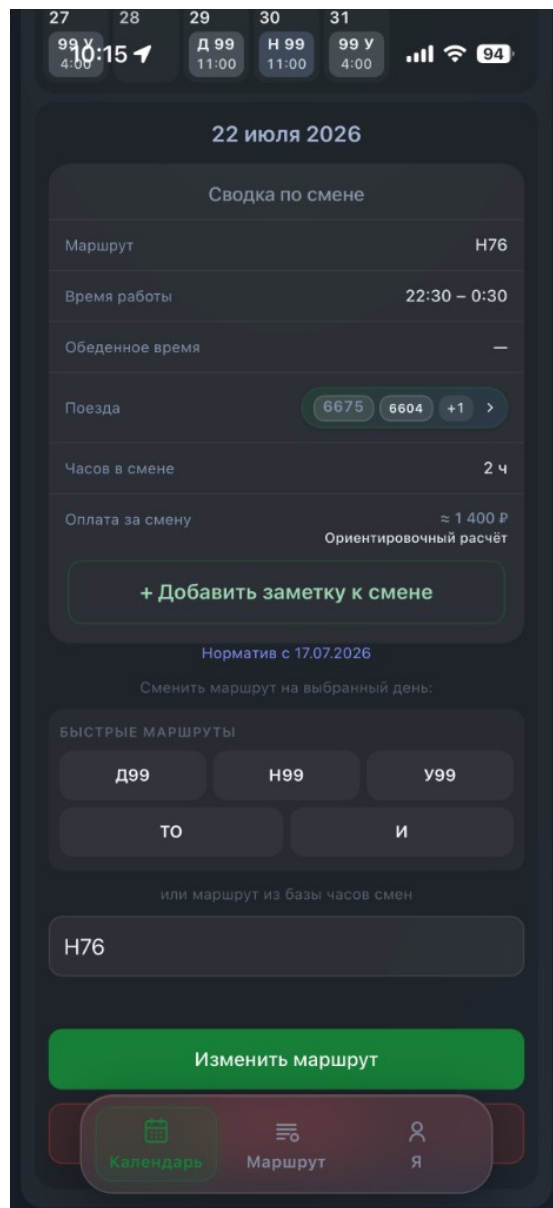


Рисунок 2 — Сводка выбранной смены и изменение маршрута.

7.3. Маршрут и расписание

Экран разделен на вкладки «Маршрут» и «Расписание». На первой задаются дата, ночной режим и до десяти поездов; результат представляется карточками со временем, остановками, пробегом, оборотами и предупреждениями. Вторая вкладка показывает таймлайн станций, live-часы, активный участок и заметки.

7.3.1. Визуальное представление маршрута и расписания

Первый экран показывает параметры смены и карточки найденных рейсов. После выбора рейса программа строит последовательный таймлайн станций с временем прибытия, отправления, километражем и предметными отметками.

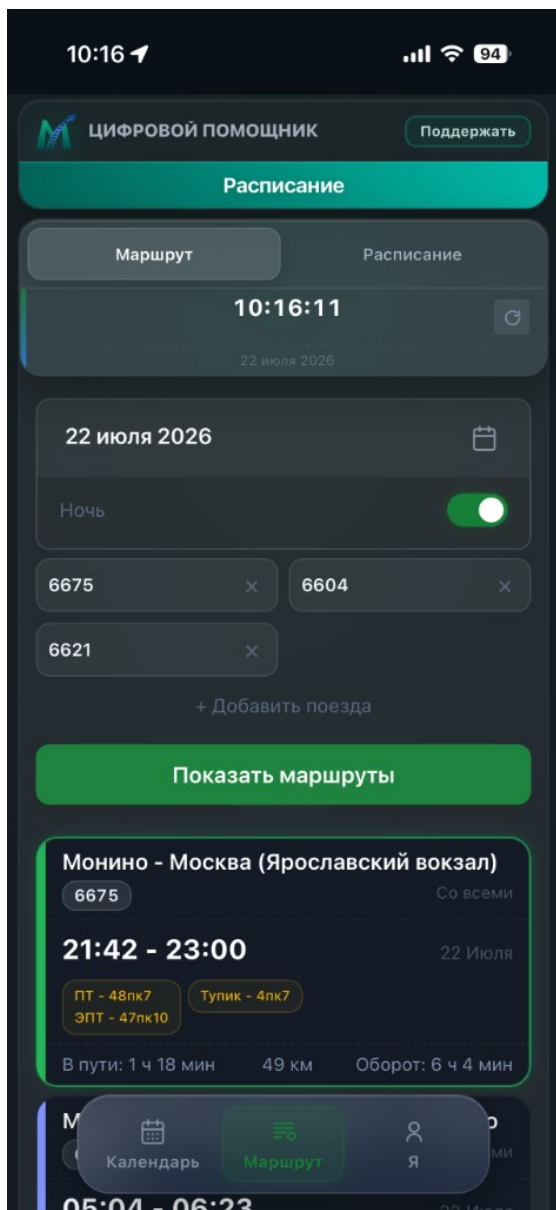


Рисунок 3 — Формирование маршрута и карточки рейсов.

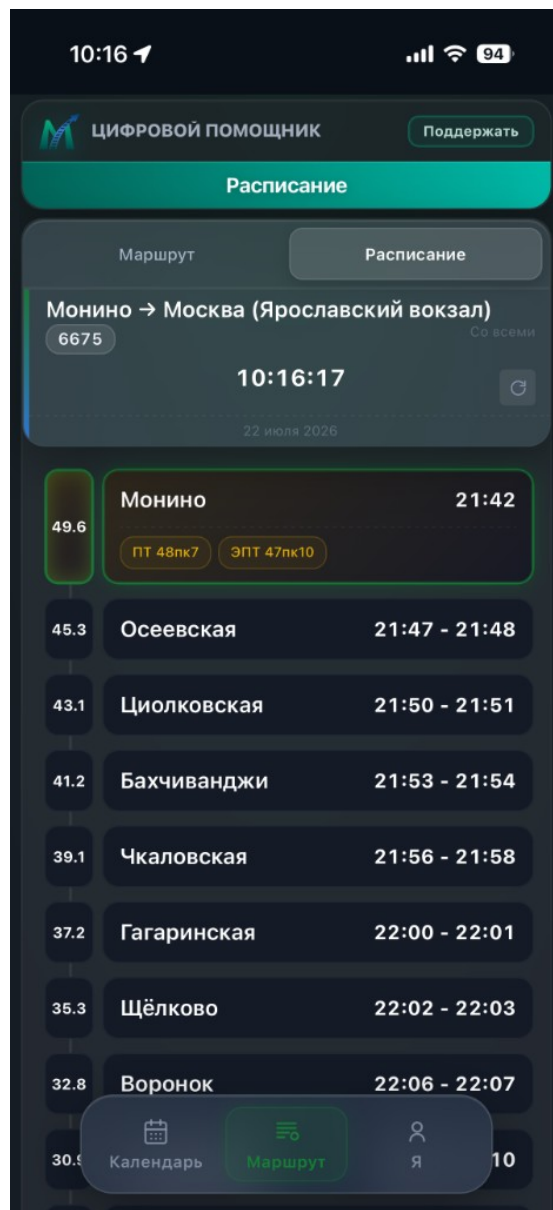


Рисунок 4 — Таймлайн расписания от начальной станции.

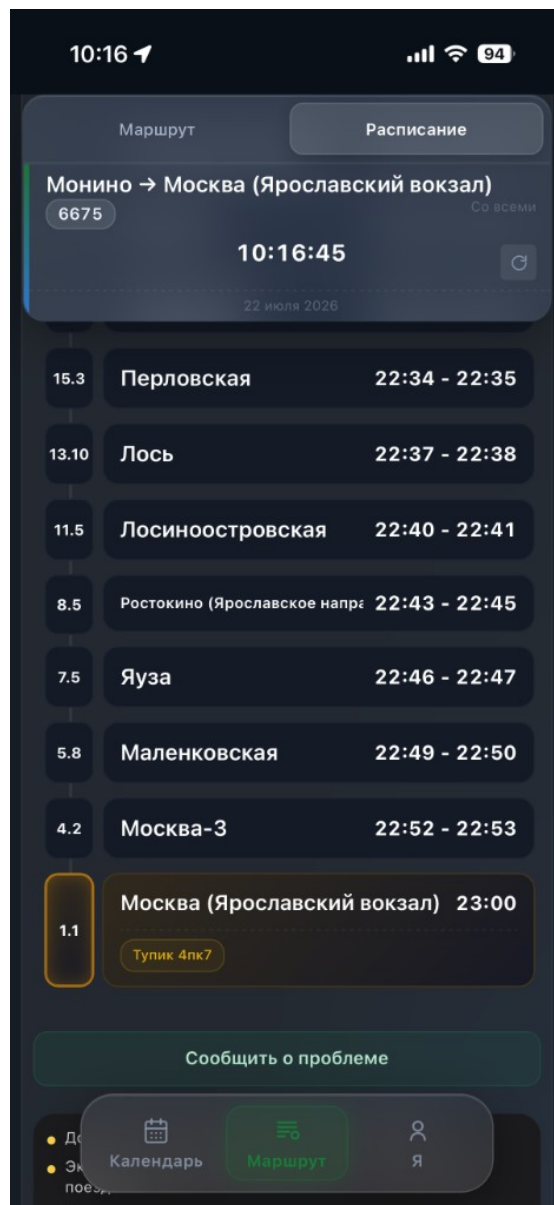


Рисунок 5 — Продолжение таймлайна до конечной станции.

7.4. Личный кабинет

Профиль содержит имя, должность и табельный номер, ближайшую смену, статистику рабочих часов и ориентировочной оплаты за выбранный месяц. Дополнительно пользователь может сохранить и открыть QR-код АСПОЖ.

7.4.1. Визуальное представление личного кабинета

Личный кабинет объединяет сведения о сотруднике, карточку ближайшей смены, финансовую и часовую статистику месяца, а также блок хранения QR-кода АСПОЖ. Персональные значения и содержимое QR-кода на иллюстрациях скрыты.

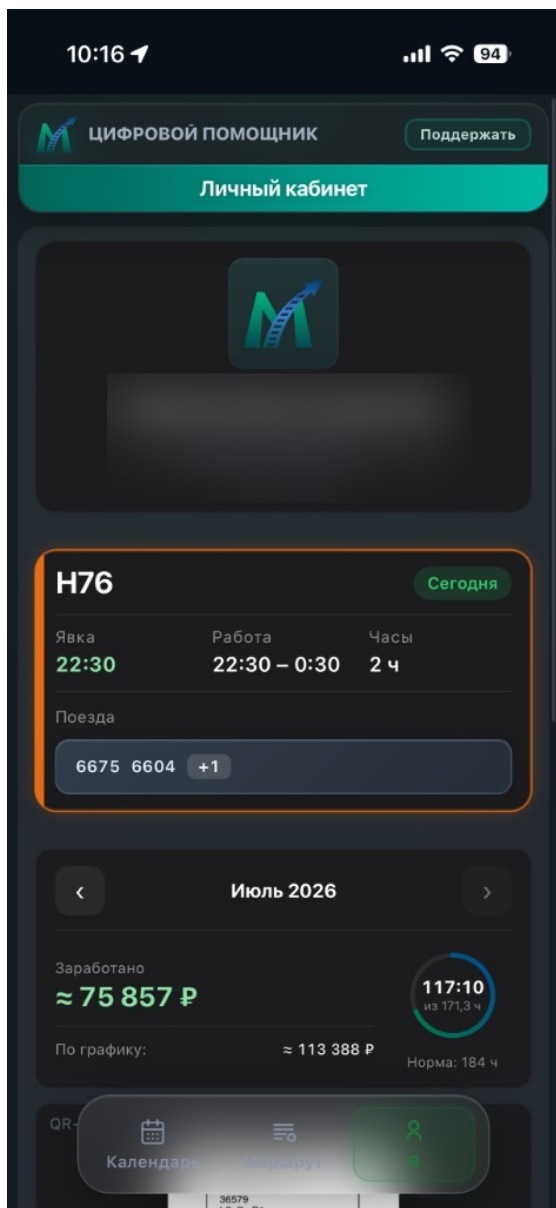


Рисунок 6 — Профиль, ближайшая смена и статистика месяца.

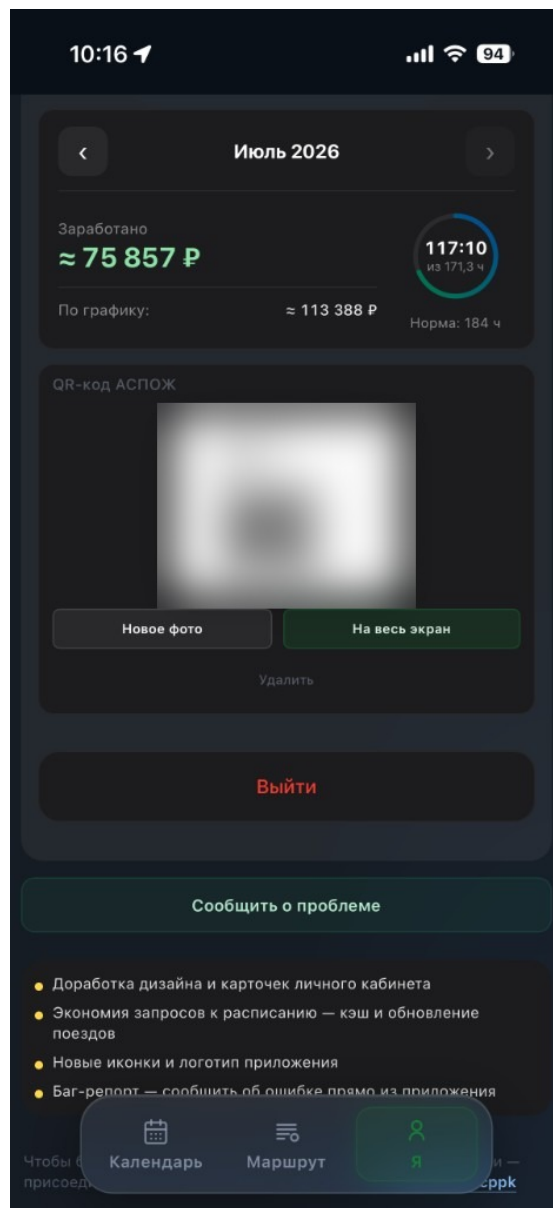


Рисунок 7 — Статистика, QR-блок АСПОЖ и управление профилем.

7.5. Общие элементы

Нижняя панель обеспечивает переходы между календарем, маршрутом и профилем. Интерфейс адаптирован для смартфонов и поддерживает свайпы. Отдельные overlay-окна используются для обновлений, заметок, предупреждений, обратной связи и подтверждения действий.

8. Состав программы и объем

Измеренный объем файлов исходного текста текущей версии: 609465 байт, 17905 строк в 13 файлах HTML, JavaScript и CSS. JSON-справочники составляют отдельный набор runtime-данных и в этот показатель не входят.

Компонент	Назначение
app/index.html	Точка входа, все экраны и основная бизнес-логика.
app/sw.js	Автономный кеш, lifecycle PWA и уведомления.
app/styles/*.css	Девять модулей оформления и адаптивности.
app/spr.json, app/trains-local.json	Предметные справочники и offline-резерв поездов.
app/data/*.json	Шаблоны смен, локальные маршруты, UID и сведения о релизах.
app/manifest.json	Метаданные устанавливаемого PWA.

9. Отбор идентифицирующих материалов

В листинг включены фрагменты, раскрывающие точку входа, персональное хранение, алгоритмы кеширования, построение маршрутов, обработку смен и Service Worker. Исключены реальные настройки внешних сервисов, бинарные изображения, исторические снимки, сторонние библиотеки, инструменты разработки и сгенерированные массивы данных.

Диапазоны строк относятся к исходникам версии 2.4.6.2 на дату формирования настоящего документа.

10. Фрагменты исходного текста

1. Разметка входа и основной навигации

Файл: app/index.html; строки 77–180. Точка входа пользовательского интерфейса: локальный вход, нижняя навигация и экран построения маршрута.

```
77 | <div id="screen-auth" class="screen user-auth-overlay" role="dialog" aria-modal="true"
    |   aria-labelledby="userAuthTitle">
78 |   <div class="user-auth-card">
79 |     <div class="user-auth-brand">
80 |       
81 |     </div>
82 |     <h1 id="userAuthTitle" class="user-auth-title">Цифровой помощник</h1>
83 |     <p class="user-auth-tagline">Быстрее к правильному действию</p>
84 |
85 |     <div class="user-auth-panels">
86 |       <div id="authPanelLogin" class="user-auth-panel active">
87 |         <p class="user-auth-subtitle">Вход в приложение</p>
88 |         <input type="text" id="authTabInput" class="user-auth-input" placeholder="Табельный номер
    |         (например 25472)" inputmode="numeric" autocomplete="username" enterkeyhint="go">
89 |         <p id="authErrorLogin" class="user-auth-error" role="alert" aria-live="polite"></p>
90 |         <button type="button" id="authLoginBtn" class="user-auth-btn">Войти</button>
91 |         <p class="user-auth-footnote">Помните: чистка кэша телефона полностью сбросит приложение
    |         и личный кабинет.</p>
92 |       </div>
93 |     </div>
94 |   </div>
95 | </div>
96 |
97 | <nav class="bottom-tabbar" aria-label="Главная навигация">
98 |   <button type="button" class="bottom-tab-btn active" data-screen="calendar"
    |   onclick="switchMainScreenAnimated('calendar')">
99 |     <span class="tab-icon" aria-hidden="true">
100 |       <svg viewBox="0 0 24 24"><rect x="3" y="4" width="18" height="18" rx="2"/><path d="M16 2v4M8
    |       2v4M3 10h18M8 14h.01M12 14h.01M16 14h.01M12 18h.01M16 18h.01"/></svg>
101 |     </span>
102 |     <span>Календарь</span>
103 |   </button>
104 |   <button type="button" class="bottom-tab-btn" data-screen="schedule"
    |   onclick="switchMainScreenAnimated('schedule')">
105 |     <span class="tab-icon" aria-hidden="true">
106 |       <svg viewBox="0 0 24 24"><path d="M4 6h16M4 10h16M4 14h16M4 18h6"/><circle cx="19" cy="17"
    |       r="3"/></svg>
107 |     </span>
108 |     <span>Маршрут</span>
109 |   </button>
110 |   <button type="button" class="bottom-tab-btn" data-screen="profile"
    |   onclick="switchMainScreenAnimated('profile')">
111 |     <span class="tab-icon" aria-hidden="true">
112 |       <svg viewBox="0 0 24 24"><circle cx="12" cy="8" r="4"/><path d="M6 20c0-3.3 2.7-6 6-6s6 2.7 6
    |       6"/></svg>
113 |     </span>
114 |     <span>Я</span>
115 |   </button>
116 | </nav>
117 |
118 | <div id="mainScreensStage" class="main-screens-stage">
119 |
120 | <div id="screen-schedule" class="screen hidden">
121 |
122 | <div class="phone-screen schedule-app-wrap">
123 |   <header class="app-screen-header">
124 |     <div class="app-header-bar">
125 |       <div class="app-header-bar-brand">
126 |         
127 |         <span class="app-brand-text">ЦИФРОВОЙ ПОМОЩНИК</span>
128 |       </div>
129 |       <button type="button" class="app-donate-trigger app-donate-trigger--header" data-donate-open
    |       aria-label="Поддержать проект">Поддержать</button>
130 |     </div>
131 |     <div class="schedule-title-plate">
132 |       <h2 class="screen-section-title">Расписание</h2>
133 |     </div>
134 |   </header>
135 |
136 |   <div class="screen-card screen-card--schedule">
137 |
138 |     <div class="schedule-glass-stack" id="scheduleGlassStack">
139 |       <div class="tabs-nav schedule-glass-tabs" aria-label="Вкладки маршрута">
140 |         <button type="button" class="tab-btn active" data-tab="tab-route"
    |         onclick="switchScheduleTabAnimated('tab-route', this)">Маршрут</button>
141 |         <button type="button" class="tab-btn" data-tab="tab-schedule"
    |         onclick="switchScheduleTabAnimated('tab-schedule', this)">Расписание</button>
142 |       </div>
143 |       <div id="scheduleHeader" class="schedule-glass-body" style="display:none;"></div>
144 |     </div>
145 |
146 |   <div id="scheduleTabsStage" class="schedule-tabs-stage">
147 |
148 |     <div id="tab-route" class="tab-content active">
149 |       <div class="shift-date-card">
150 |         <div class="shift-date-main">
151 |           <label class="shift-date-picker" id="datePickerLabel" title="Выбрать дату смены">
```

```

152 |         <span id="dateDisplayText" class="shift-date-display"></span>
153 |         <input type="date" id="dateInput" class="shift-date-input" aria-label="Дата смены">
154 |         <span class="shift-date-calendar-icon" aria-hidden="true">
155 |             <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="1.75" stroke-linecap="round" stroke-linejoin="round">
156 |                 <rect x="3" y="4" width="18" height="18" rx="2"></rect>
157 |                 <path d="M16 2v4M8 2v4M3 10h18"></path>
158 |             </svg>
159 |         </span>
160 |     </label>
161 | </div>
162 |     <div class="shift-date-night">
163 |         <span class="shift-date-night-label">Ночь</span>
164 |         <label class="night-switch">
165 |             <input type="checkbox" id="nightMode">
166 |             <span class="night-switch-slider"></span>
167 |         </label>
168 |     </div>
169 | </div>
170 | <div class="trains-dynamic">
171 |     <div id="trainsList" class="trains-list"></div>
172 |     <button type="button" id="addTrainBtn" class="add-train-btn">+ Добавить поезда</button>
173 | </div>
174 | <div class="trains-submit">
175 |     <button class="action-btn" onclick="getRouteData()">Показать маршруты</button>
176 | </div>
177 |
178 | <div id="routeLoader" class="loader">Получение данных...</div>
179 |
180 | <div id="routeGrid" class="route-grid" style="display: none;"></div>

```

2. Конфигурация ядра и персональное локальное хранение

Файл: app/index.html; строки 483–624. Инициализация предметных констант, offline-состояния и изоляция данных пользователей по табельному номеру.

```

483 | <script>
484 |     const APP_CFG = window.APP_CONFIG || {};
485 |
486 |     const DASH_CHAR = String.fromCharCode(45);
487 |     const API_KEY = APP_CFG.yandexApiKey || '';
488 |     const API_BASE = '[ВНЕШНИЙ_УРЛ_ИСКЛЮЧЕН]';
489 |     const MY_CLOUDFLARE_PROXY = APP_CFG.yandexProxy || '';
490 |     const ANCHOR_STATION = 's2000002';
491 |     const EXTRA_SCHEDULE_STATIONS = ['s9600701'];
492 |     // Логика запросов поездов: app/data/trains-api/README.md (лимит ~500/сутки)
493 |     const stationScheduleByKey = {};
494 |     const stationUidIndexByDate = {};
495 |     const stationScheduleLoaders = {};
496 |
497 |     const STATION_BOARD_CACHE_VERSION = 2;
498 |     const STATION_BOARD_CACHE_MAX_DATES = 4;
499 |     const ROUTE_REFRESH_LONG_PRESS_MS = 580;
500 |
501 |     let stationScheduleCache = { version: STATION_BOARD_CACHE_VERSION, boards: {} };
502 |     let trainThreadsCache = {};
503 |     let trainUidStaticIndex = null;
504 |     let sprDataCache = null;
505 |     let localTrainsCache = null;
506 |
507 |     const USERS_DB_KEY = 'rzd_users';
508 |     const AUTH_SESSION_KEY = 'currentUserTab';
509 |     const DEPOT_SHIFTS_KEY = 'depotShifts';
510 |     const REFERENCE_SPR_STORAGE_KEY = 'cppk_spr_cache';
511 |     const REFERENCE_TRAINS_STORAGE_KEY = 'cppk_trains_local_cache';
512 |     const CALENDAR_LOCAL_ROUTES_STORAGE_KEY = 'cppk_calendar_local_routes';
513 |     const LOCAL_SHIFT_TEMPLATES_URL = './data/shift-templates.json';
514 |     const LOCAL_SHIFT_TEMPLATES_STORAGE_KEY = 'cppk_shift_templates_bundle';
515 |     const DEPOT_FIO_KEY = 'depotFio';
516 |     const DEPOT_TAB_KEY = 'depotTab';
517 |     const DEPOT_POSITION_KEY = 'depotPosition';
518 |     const APP_LAST_SEEN_VERSION_KEY = 'aplastSeenVersion';
519 |     const APP_RELEASE_VERSION = '2.4.6.2';
520 |     const DEPOT_API_URL = APP_CFG.depotApiUrl || '';
521 |     const FEEDBACK_TELEGRAM_BOT_URL = '[ВНЕШНИЙ_УРЛ_ИСКЛЮЧЕН]';
522 |     const APP_TELEGRAM_URL = '[ВНЕШНИЙ_УРЛ_ИСКЛЮЧЕН]';
523 |     let calendarTcsExportInProgress = false;
524 |
525 |     function getDonateConfig() {
526 |         return {
527 |             layoutId: APP_CFG.cloudtipsLayoutId || '',
528 |             paymentPageUrl: APP_CFG.cloudtipsPaymentUrl || '',
529 |             feeApiUrl: APP_CFG.cloudtipsFeeApiUrl || '[ВНЕШНИЙ_УРЛ_ИСКЛЮЧЕН]'
530 |         };
531 |     }
532 |
533 |     const ROUTE_FETCH_TIMEOUT_MS = 22000;
534 |     let routeDataRequestId = 0;
535 |
536 |     async function fetchWithTimeout(url, options = {}, timeoutMs = ROUTE_FETCH_TIMEOUT_MS) {
537 |         const controller = new AbortController();
538 |         const timer = window.setTimeout(() => controller.abort(), timeoutMs);
539 |         try {
540 |             return await fetch(url, { ...options, signal: controller.signal });
541 |         } catch (error) {
542 |             if (error?.name === 'AbortError') {
543 |                 throw new Error(`Превышено время ожидания (${Math.round(timeoutMs / 1000)} с)`);
544 |             }
545 |         }
546 |     }

```

```

545 |         throw error;
546 |     } finally {
547 |         window.clearTimeout(timer);
548 |     }
549 | }
550 |
551 | function hasLocalUserData(tabNumber) {
552 |     const tab = normalizeTabNumber(tabNumber);
553 |     if (!tab) return false;
554 |     if (hasLocalShiftTemplates()) return true;
555 |     const shiftsRaw = localStorage.getItem(userShiftsStorageKey(tab));
556 |     if (!shiftsRaw) return false;
557 |     const shifts = parseJSON(shiftsRaw, null);
558 |     return shifts && typeof shifts === 'object' && Object.keys(shifts).length > 0;
559 | }
560 |
561 | function isAppOffline() {
562 |     return typeof navigator !== 'undefined' && navigator.onLine === false;
563 | }
564 |
565 | function persistReferenceJsonCache(storageKey, data) {
566 |     if (!storageKey || data == null) return;
567 |
568 |     try {
569 |         localStorage.setItem(storageKey, JSON.stringify(data));
570 |     } catch (err) {
571 |         console.warn('[offline-cache] не удалось сохранить', storageKey, err);
572 |     }
573 |
574 |     function loadReferenceJsonCache(storageKey) {
575 |         if (!storageKey) return null;
576 |         const raw = localStorage.getItem(storageKey);
577 |         if (raw == null) return null;
578 |         try {
579 |             return JSON.parse(raw);
580 |         } catch (err) {
581 |             console.warn('[offline-cache] повреждён кэш', storageKey, err);
582 |             return null;
583 |         }
584 |     }
585 |
586 |     function syncOfflineModeUi() {
587 |         document.body.classList.toggle('app-offline', isAppOffline());
588 |     }
589 |
590 |     function notifyOfflineModeIfNeeded() {
591 |         if (!isAppOffline()) return;
592 |         if (typeof showToast === 'function') {
593 |             showToast('Автономный режим – данные из памяти устройства');
594 |         }
595 |     }
596 |
597 |     function getCurrentUserTab() {
598 |         return (localStorage.getItem(AUTH_SESSION_KEY) || '').trim();
599 |     }
600 |
601 |     function userStorageKey(key) {
602 |         const tab = getCurrentUserTab();
603 |         return tab ? `u_${tab}_${key}` : key;
604 |     }
605 |
606 |     function userStorageGet(key, fallback = null) {
607 |         const raw = localStorage.getItem(userStorageKey(key));
608 |         return raw == null ? fallback : raw;
609 |     }
610 |
611 |     function userStorageGetJson(key, fallback) {
612 |         const raw = localStorage.getItem(userStorageKey(key));
613 |         if (raw == null) return fallback;
614 |         try { return JSON.parse(raw); } catch (e) { return fallback; }
615 |     }
616 |
617 |     function userStorageSet(key, value) {
618 |         const str = typeof value === 'string' ? value : JSON.stringify(value);
619 |         localStorage.setItem(userStorageKey(key), str);
620 |     }
621 |
622 |     function userStorageRemove(key) {
623 |         localStorage.removeItem(userStorageKey(key));
624 |     }

```

3. Миграция кеша поездов и offline fallback справочников

Файл: app/index.html; строки 2027–2173. Нормализация ключей «номер@дата», сериализация ниток и загрузка статических JSON с резервом из localStorage.

```

2027 |     function migrateTrainThreadsCache(cache) {
2028 |         const source = cache && typeof cache === 'object' ? cache : {};
2029 |         const migrated = {};
2030 |
2031 |         Object.entries(source).forEach(([key, value]) => {
2032 |             if (!value || typeof value !== 'object') return;
2033 |
2034 |             if (key.includes('@')) {
2035 |                 const atIdx = key.indexOf('@');
2036 |                 const numPart = normalizeTrainCacheKey(key.slice(0, atIdx));
2037 |                 const datePart = String(key.slice(atIdx + 1) || '').trim();
2038 |                 if (numPart && datePart) {

```

```

2039 |         migrated[`${numPart}@${datePart}`] = {
2040 |             ...value,
2041 |             scheduleDate: value.scheduleDate || datePart
2042 |         };
2043 |     }
2044 |     return;
2045 | }
2046 |
2047 | const numKey = normalizeTrainCacheKey(key);
2048 | if (!numKey) return;
2049 |
2050 | const date = value.scheduleDate || null;
2051 | if (date) {
2052 |     migrated[`${numKey}@${date}`] = { ...value, scheduleDate: date };
2053 | } else {
2054 |     migrated[numKey] = value;
2055 | }
2056 | });
2057 |
2058 | return migrated;
2059 | }
2060 |
2061 | function loadTrainThreadsCacheFromStorage() {
2062 |     const raw = userStorageGet('trainThreadsCache');
2063 |     if (!raw) {
2064 |         trainThreadsCache = {};
2065 |         return;
2066 |     }
2067 |
2068 |     const parsed = typeof raw === 'string' ? parseJSON(raw) : raw;
2069 |
2070 |     trainThreadsCache = migrateTrainThreadsCache(parsed);
2071 |     userStorageSet('trainThreadsCache', trainThreadsCache);
2072 | }
2073 |
2074 | function serializeTrainCacheEntry(result, scheduleDate) {
2075 |     if (!result) return null;
2076 |     const date = scheduleDate || result.scheduleDate || null;
2077 |     return {
2078 |         uid: result.uid || '',
2079 |         number: result.number || '',
2080 |         title: result.title || '',
2081 |         departure: result.departure || '',
2082 |         arrival: result.arrival || '',
2083 |         duration: result.duration ?? null,
2084 |         stops: Array.isArray(result.stops) ? result.stops : [],
2085 |         stops_text: result.stops_text || resolveStopsText('', result.stops),
2086 |         scheduleDate: date,
2087 |         serviceDays: result.serviceDays || null,
2088 |         source: result.source || 'cache',
2089 |         runsOnSelectedDate: result.runsOnSelectedDate,
2090 |         scheduleDayWarning: result.scheduleDayWarning || ''
2091 |     };
2092 | }
2093 |
2094 | async function fetchJsonAsset(paths, storageKey = null) {
2095 |     const candidates = Array.isArray(paths) ? paths : [paths];
2096 |     let lastError = null;
2097 |
2098 |     for (const path of candidates) {
2099 |         try {
2100 |             const res = await fetch(path);
2101 |             if (!res.ok) {
2102 |                 lastError = new Error(`HTTP ${res.status} для ${path}`);
2103 |                 continue;
2104 |             }
2105 |             const text = await res.text();
2106 |             if (!text.trim()) {
2107 |                 lastError = new Error(`Файл пуст: ${path}`);
2108 |                 continue;
2109 |             }
2110 |             const parsed = JSON.parse(text);
2111 |             persistReferenceJsonCache(storageKey, parsed);
2112 |
2113 |             return parsed;
2114 |         } catch (err) {
2115 |             lastError = err;
2116 |         }
2117 |     }
2118 |
2119 |     const cached = loadReferenceJsonCache(storageKey);
2120 |     if (cached) {
2121 |         console.warn(`[offline-cache] JSON из localStorage:`, storageKey || candidates[0]);
2122 |         return cached;
2123 |     }
2124 |
2125 |     throw lastError || new Error('Не удалось загрузить JSON');
2126 | }
2127 |
2128 | async function loadReferenceData() {
2129 |     const basePaths = ['./spr.json', 'spr.json', 'app/spr.json'];
2130 |     const trainPaths = ['./trains-local.json', 'trains-local.json', 'app/trains-local.json'];
2131 |
2132 |     try {
2133 |         sprDataCache = await fetchJsonAsset(basePaths, REFERENCE_SPR_STORAGE_KEY);
2134 |         console.log('Справочник успешно загружен:', sprDataCache);
2135 |     } catch (err) {
2136 |         const cachedSpr = loadReferenceJsonCache(REFERENCE_SPR_STORAGE_KEY);
2137 |         if (cachedSpr) {
2138 |             sprDataCache = cachedSpr;
2139 |             console.warn('Справочник из автономного кэша (spr.json недоступен)');
2140 |         } else if (window.location.protocol === 'file:') {

```

```

2139 |         alert('Не удалось загрузить spr.json. Открой приложение через локальный сервер: npm run
serve ([ВНЕШНИЙ_URL_ИСКЛЮЧЕН]);
2140 |         throw err;
2141 |     } else if (err?.name === 'SyntaxError') {
2142 |         alert('Файл spr.json содержит ошибку (невалидный JSON). Проверь и сохрани файл
(Ctrl+S).');
2143 |         throw err;
2144 |     } else {
2145 |         alert('Не удалось загрузить spr.json. Нужен интернет для первого входа или перезагрузите
после работы онлайн.');
```

```

2146 |         throw err;
2147 |     }
2148 | }
2149 |
2150 |     try {
2151 |         localTrainsCache = await fetchJsonAsset(trainPaths, REFERENCE_TRAINS_STORAGE_KEY);
2152 |         console.log('Локальная база поездов загружена:', localTrainsCache?.trains?.length || 0);
2153 |     } catch (err) {
2154 |         const cachedTrains = loadReferenceJsonCache(REFERENCE_TRAINS_STORAGE_KEY);
2155 |         if (cachedTrains) {
2156 |             localTrainsCache = cachedTrains;
2157 |             console.warn('Локальная база поездов из автономного кэша');
```

```

2158 |         } else {
2159 |             console.warn('Локальная база поездов недоступна:', err);
2160 |             localTrainsCache = null;
2161 |         }
2162 |     }
2163 |
2164 |     const uidPaths = ['./data/trains-uids.json', 'data/trains-uids.json',
'app/data/trains-uids.json'];
2165 |     try {
2166 |         const uidBundle = await fetchJsonAsset(uidPaths, 'cppk_trains_uids_cache');
```

```

2167 |         trainUidStaticIndex = uidBundle?.uids && typeof uidBundle.uids === 'object' ? uidBundle.uids
: null;
2168 |         if (trainUidStaticIndex) {
2169 |             console.log('[trains] uid index:', Object.keys(trainUidStaticIndex).length);
2170 |         }
2171 |     } catch (err) {
2172 |         trainUidStaticIndex = loadReferenceJsonCache('cppk_trains_uids_cache')?.uids || null;
2173 |     }

```

4. Формирование последовательности станций и расчет маршрута

Файл: app/index.html; строки 3520–3647. Сопоставление фактических остановок с эталонным маршрутом, размещение предупреждений и расчет пробега.

```

3520 |     function buildScheduledDisplayStops(threadStops) {
3521 |         if (!Array.isArray(threadStops) || !threadStops.length) return [];
3522 |
3523 |         const referenceList = getRouteReferenceList(threadStops);
3524 |         if (!referenceList?.length) {
3525 |             return threadStops.map(stop => ({
3526 |                 referenceTitle: stop.station?.title || 'Станция',
3527 |                 threadStop: stop,
3528 |                 passThrough: false
3529 |             }));
3530 |         }
3531 |
3532 |         const firstTitle = threadStops[0]?.station?.title || '';
3533 |         const lastTitle = threadStops[threadStops.length - 1]?.station?.title || '';
3534 |         const firstIdx = referenceIndexForStation(referenceList, firstTitle);
3535 |         const lastIdx = referenceIndexForStation(referenceList, lastTitle);
3536 |
3537 |         if (firstIdx === -1 || lastIdx === -1) {
3538 |             return threadStops.map(stop => ({
3539 |                 referenceTitle: stop.station?.title || 'Станция',
3540 |                 threadStop: stop,
3541 |                 passThrough: false
3542 |             }));
3543 |         }
3544 |
3545 |         const start = Math.min(firstIdx, lastIdx);
3546 |         const end = Math.max(firstIdx, lastIdx);
3547 |         const usedThreadIndices = new Set();
3548 |         let threadCursor = 0;
3549 |
3550 |         return referenceList.slice(start, end + 1).map(refTitle => {
3551 |             let threadStop = null;
3552 |
3553 |             for (let i = threadCursor; i < threadStops.length; i++) {
3554 |                 if (usedThreadIndices.has(i)) continue;
3555 |
3556 |                 const threadTitle = threadStops[i].station?.title || '';
3557 |                 if (!stationsMatchStrict(refTitle, threadTitle)) continue;
3558 |
3559 |                 threadStop = threadStops[i];
3560 |                 usedThreadIndices.add(i);
3561 |                 threadCursor = i + 1;
```

```

3562 |             break;
3563 |         }
3564 |
3565 |         return {
3566 |             referenceTitle: refTitle,
3567 |             threadStop: threadStop || null,
3568 |             passThrough: !threadStop
3569 |         };
3570 |     });
3571 | }

```

```

3572 |
3573 |     function deriveStopsLabelFromReference(threadStops) {
3574 |         const displayStops = buildScheduleDisplayStops(threadStops);
3575 |         if (!displayStops.length) return 'Без остановок';
3576 |
3577 |         const allPresent = displayStops.every(item => !item.passThrough);
3578 |         return allPresent ? 'Со всеми' : 'Без остановок';
3579 |     }
3580 |
3581 |     function warningKmToFloat(warn) {
3582 |         const km = parseFloat(warn.km);
3583 |         const pk = parseFloat(warn.pk || 0);
3584 |         if (isNaN(km)) return null;
3585 |         return km + (isNaN(pk) ? 0 : pk / 10);
3586 |     }
3587 |
3588 |     function findWarningDisplayStationIndex(warnKmFloat, stopChainages, trainDir) {
3589 |         for (let i = 1; i < stopChainages.length; i++) {
3590 |             const prevKm = stopChainages[i - 1];
3591 |             const currKm = stopChainages[i];
3592 |             if (prevKm === null || currKm === null) continue;
3593 |
3594 |             if (trainDir === 'out') {
3595 |                 if (warnKmFloat > prevKm && warnKmFloat < currKm) return i - 1;
3596 |             } else if (warnKmFloat < prevKm && warnKmFloat > currKm) {
3597 |                 return i - 1;
3598 |             }
3599 |         }
3600 |         return -1;
3601 |     }
3602 |
3603 |     function formatShiftWarningPlaqueText(warn) {
3604 |
3605 |         const km = String(warn.km || '').trim();
3606 |         const pk = String(warn.pk || '').trim();
3607 |         const speed = String(warn.speed || '').trim();
3608 |         let location = '';
3609 |         if (km) location += `${km} км`;
3610 |         if (pk) location += location ? ` ${pk} нк` : ` ${pk} нк`;
3611 |         if (!location) location = '-';
3612 |         return `Скорость ${speed} км/ч на ${location}`;
3613 |     }
3614 |
3615 |     function buildWarningsByStationIndex(stops, trainDir) {
3616 |         const stopChainages = stops.map(stop => stationChainage(stop.station?.title || ''));
3617 |         const warningsByStation = {};
3618 |
3619 |         loadShiftWarnings()
3620 |             .filter(warn => String(warn.speed || '').trim() && warn.dir === trainDir)
3621 |             .forEach(warn => {
3622 |                 const warnKmFloat = warningKmToFloat(warn);
3623 |                 if (warnKmFloat === null) return;
3624 |
3625 |                 const displayIdx = findWarningDisplayStationIndex(warnKmFloat, stopChainages, trainDir);
3626 |                 if (displayIdx < 0) return;
3627 |
3628 |                 const text = formatShiftWarningPlaqueText(warn);
3629 |                 if (!warningsByStation[displayIdx]) warningsByStation[displayIdx] = [];
3630 |                 warningsByStation[displayIdx].push(text);
3631 |             });
3632 |
3633 |         return warningsByStation;
3634 |     }
3635 |
3636 |     function calcRouteMileageKm(firstTitle, lastTitle) {
3637 |         const start = stationChainage(firstTitle);
3638 |         const end = stationChainage(lastTitle);
3639 |         if (start === null || end === null) return null;
3640 |         return Math.round(Math.abs(end - start));
3641 |     }
3642 |
3643 |     function calcTurnaroundMileageKm(endTitle, nextStartTitle) {
3644 |         const endPos = stationChainage(endTitle);
3645 |         const startPos = stationChainage(nextStartTitle);
3646 |         if (endPos === null || startPos === null) return null;
3647 |
3648 |         return Math.round(Math.abs(startPos - endPos));

```

5. Cache-first поиск поезда

Файл: app/index.html; строки 3700–3865. Поиск нитки поезда с приоритетом локального кеша, статического UID, станционного табло и сетевого запроса.

```

3700 |     async function searchTrainByNumber(trainNumber, scheduleDate, options = {}) {
3701 |         const forceRefresh = options.forceRefresh === true;
3702 |         const fromCacheOnly = options.fromCacheOnly === true;
3703 |         const threadsOnly = options.threadsOnly === true;
3704 |         const registryEntry = findLocalTrainRegistryEntry(trainNumber);
3705 |
3706 |         try {
3707 |             const date = scheduleDate || getSelectedDate();
3708 |
3709 |             if (!forceRefresh) {
3710 |                 const cachedEntry = findCachedTrainResult(trainNumber, date);
3711 |                 if (cachedEntry) {
3712 |                     console.log('[trains] cache hit:', trainNumber, date);
3713 |                     const cached = decorateTrainSearchResult(
3714 |                         { ...cachedEntry },
3715 |                         trainNumber,

```

```

3716 |         date,
3717 |         registryEntry
3718 |     );
3719 |     return {
3720 |         ...cached,
3721 |         scheduleDate: date,
3722 |         stops_text: resolveStopsText('', cached.stops)
3723 |     };
3724 |     }
3725 | }
3726 |
3727 | if (fromCacheOnly) {
3728 |     const localEntry = findLocalTrainEntry(trainNumber, date) || registryEntry;
3729 |     if (localEntry) {
3730 |         const localResult = buildTrainResultFromLocalEntry(localEntry, date);
3731 |         const decorated = decorateTrainSearchResult(localResult, trainNumber, date,
localEntry);
3732 |         cacheTrainThread(trainNumber, decorated, date);
3733 |         console.log('[trains] local cache fill:', trainNumber, date);
3734 |         return decorated;
3735 |     }
3736 |     return null;
3737 | }
3738 |
3739 | const threadUid = resolveThreadUidForTrain(trainNumber, date);
3740 | if (threadUid) {
3741 |     console.log('[trains] thread by uid:', trainNumber, threadUid);
3742 |     const threadResult = await fetchTrainByThreadUid(threadUid, trainNumber, date);
3743 |     if (threadResult) {
3744 |         const result = decorateTrainSearchResult(threadResult, trainNumber, date,
registryEntry);
3745 |         cacheTrainThread(trainNumber, result, date);
3746 |         return result;
3747 |     }
3748 | }
3749 |
3750 | if (!threadsOnly) {
3751 |     const trainData = await findTrainOnStationBoards(trainNumber, date, options);
3752 |     if (trainData) {
3753 |         const result = await fetchThreadDetails(trainData, date);
3754 |         if (result) {
3755 |             const decorated = decorateTrainSearchResult(result, trainNumber, date,
registryEntry);
3756 |             cacheTrainThread(trainNumber, decorated, date);
3757 |             return decorated;
3758 |         }
3759 |     }
3760 | }
3761 |
3762 | const localEntry = findLocalTrainEntry(trainNumber, date);
3763 | if (localEntry) {
3764 |     const localResult = buildTrainResultFromLocalEntry(localEntry, date);
3765 |     const decorated = decorateTrainSearchResult(localResult, trainNumber, date, localEntry);
3766 |     cacheTrainThread(trainNumber, decorated, date);
3767 |     console.log('[trains] local fallback:', trainNumber);
3768 |     return decorated;
3769 | }
3770 |
3771 | if (registryEntry) {
3772 |     const localResult = buildTrainResultFromLocalEntry(registryEntry, date);
3773 |     const decorated = decorateTrainSearchResult(localResult, trainNumber, date,
registryEntry);
3774 |     cacheTrainThread(trainNumber, decorated, date);
3775 |     return decorated;
3776 | }
3777 |
3778 | console.warn(`[trains] not found: ${trainNumber} @ ${date}`);
3779 | return null;
3780 | } catch (error) {
3781 |     console.error('[trains]', error);
3782 |     if (!fromCacheOnly) {
3783 |         const fallbackDate = scheduleDate || getSelectedDate();
3784 |         const threadUid = resolveThreadUidForTrain(trainNumber, fallbackDate);
3785 |         if (threadUid) {
3786 |             try {
3787 |                 const threadResult = await fetchTrainByThreadUid(threadUid, trainNumber,
fallbackDate);
3788 |                 if (threadResult) {
3789 |                     const result = decorateTrainSearchResult(
threadResult,
3790 |                         trainNumber,
3791 |                         fallbackDate,
3792 |                         registryEntry
3793 |                     );
3794 |                     cacheTrainThread(trainNumber, result, fallbackDate);
3795 |                     return result;
3796 |                 }
3797 |             } catch (threadErr) {
3798 |                 console.warn('[trains] uid fallback failed:', threadErr);
3799 |             }
3800 |         }
3801 |     }
3802 |
3803 |     const localEntry = findLocalTrainEntry(trainNumber, fallbackDate) || registryEntry;
3804 |     if (localEntry) {
3805 |         const localResult = buildTrainResultFromLocalEntry(localEntry, fallbackDate);
3806 |         const decorated = decorateTrainSearchResult(localResult, trainNumber, fallbackDate,
localEntry);
3807 |         cacheTrainThread(trainNumber, decorated, fallbackDate);
3808 |         return decorated;
3809 |     }

```



```

3810 |         }
3811 |         return null;
3812 |     }
3813 | }
3814 |
3815 | function resolveShiftTrainPlanForInputs(inputs) {
3816 |     const stored = userStorageGetJson('shiftTrainPlan', null);
3817 |     if (!stored || !Array.isArray(stored.items) || !Array.isArray(inputs)) return null;
3818 |
3819 |     const inputKeys = inputs.map(normalizeTrainCacheKey);
3820 |
3821 |     const storedKeys = stored.items.map(item => normalizeTrainCacheKey(item.number));
3822 |     if (inputKeys.length !== storedKeys.length) return null;
3823 |
3824 |     for (let i = 0; i < inputKeys.length; i++) {
3825 |         if (inputKeys[i] !== storedKeys[i]) return null;
3826 |     }
3827 |     return stored.items;
3828 | }
3829 |
3830 | function collectBoardDatesForShift(trainPlan, baseDate) {
3831 |     const dates = new Set();
3832 |     if (baseDate) dates.add(baseDate);
3833 |     if (Array.isArray(trainPlan)) {
3834 |         trainPlan.forEach(item => {
3835 |             if (item?.scheduleDate) dates.add(item.scheduleDate);
3836 |             if (item?.morningScheduleDate) dates.add(item.morningScheduleDate);
3837 |         });
3838 |     } else if (isNightMode() && baseDate) {
3839 |         dates.add(addDaysToDateStr(baseDate, 1));
3840 |     }
3841 |     return [...dates];
3842 | }
3843 |
3844 | async function searchTrainByNumberSmart(trainNumber, options = {}) {
3845 |     const planEntry = Array.isArray(options.trainPlan)
3846 |         ? options.trainPlan.find(item => normalizeTrainCacheKey(item.number) ===
3847 |           normalizeTrainCacheKey(trainNumber))
3848 |         : null;
3849 |
3850 |     if (planEntry?.scheduleDate) {
3851 |         if (planEntry.morningScheduleDate && planEntry.segment === 'both') {
3852 |             const eveningTrain = await searchTrainByNumber(trainNumber, planEntry.scheduleDate,
3853 |               options);
3854 |             const morningTrain = await searchTrainByNumber(trainNumber,
3855 |               planEntry.morningScheduleDate, options);
3856 |             if (morningTrain && isMorningNextShiftDeparture(morningTrain.departure)) {
3857 |                 return morningTrain;
3858 |             }
3859 |             if (eveningTrain && !isMorningNextShiftDeparture(eveningTrain.departure)) {
3860 |                 return eveningTrain;
3861 |             }
3862 |             return morningTrain || eveningTrain;
3863 |         }
3864 |         return searchTrainByNumber(trainNumber, planEntry.scheduleDate, options);
3865 |     }
3866 | }
3867 |
3868 | const baseDate = getSelectedDate();
3869 | if (!isNightMode()) {
3870 |     return searchTrainByNumber(trainNumber, baseDate, options);
3871 | }

```

6. Оркестрация построения маршрута

Файл: app/index.html; строки 4068–4205. Параллельное получение поездов, защита от устаревших запросов и расчет времени, оборотов и суммарного пробега.

```

4068 |     async function getRouteData(options = {}) {
4069 |         const inputs = getTrainNumbersFromDom();
4070 |         const forceRefresh = options.forceRefresh === true;
4071 |         const fromCacheOnly = options.fromCacheOnly === true;
4072 |         const threadsOnly = options.threadsOnly === true;
4073 |         const forceBoardRefresh = options.forceBoardRefresh === true;
4074 |         const trainPlan = resolveShiftTrainPlanForInputs(inputs);
4075 |         const searchOptions = { forceRefresh, fromCacheOnly, threadsOnly, forceBoardRefresh, trainPlan };
4076 |         const baseDate = getSelectedDate();
4077 |
4078 |         if (inputs.length === 0) {
4079 |             alert('Введите хотя бы один номер поезда.');
```

```

4101 |     ensureScheduleNotificationPermission();
4102 |     userStorageSet('trainNumbers', inputs);
4103 |     syncScheduleLiveClock();
4104 |
4105 |     if (!fromCacheOnly) {
4106 |         loader.style.display = 'flex';
4107 |         grid.style.display = 'none';
4108 |         refreshBtn?.classList.add('is-loading');
4109 |     }
4110 |
4111 |     setRouteToolBarVisible(false);
4112 |     setRouteSummaryVisible(false);
4113 |
4114 |     try {
4115 |         let boardWarningHtml = '';
4116 |
4117 |         if (!fromCacheOnly && !threadsOnly) {
4118 |             const boardDates = collectBoardDatesForShift(trainPlan, baseDate);
4119 |             const boardResults = await ensureStationBoardsForDates(boardDates, {
4120 |                 forceRefresh: forceRefresh || forceBoardRefresh
4121 |             });
4122 |             if (requestId !== routeDataRequestId) return;
4123 |
4124 |             const failedBoards = boardResults.filter(item => !item.ok);
4125 |             if (failedBoards.length && failedBoards.length === boardResults.length) {
4126 |                 boardWarningHtml = '<div class="route-api-warning" style="width:100%;padding:10px 12px;margin-bottom:8px;border-radius:8px;background:rgba(255,107,107,0.12);color:#ffb4b4;font-size:13px;">Не удалось загрузить расписание с вокзала. Показаны кэш и локальные данные.</div>';
4127 |             } else if (failedBoards.length) {
4128 |                 boardWarningHtml = '<div class="route-api-warning" style="width:100%;padding:10px 12px;margin-bottom:8px;border-radius:8px;background:rgba(140,148,160,0.12);color:rgba(210,216,224,0.92);font-size:13px;">Часть данных с вокзала недоступна. Показаны кэш и локальные данные.</div>';
4129 |             }
4130 |
4131 |             const trainPromises = inputs.map(async trainNumber => {
4132 |                 try {
4133 |                     return await searchTrainByNumberSmart(trainNumber, searchOptions);
4134 |                 } catch (err) {
4135 |                     console.error('[getRouteData] train failed:', trainNumber, err);
4136 |                     return null;
4137 |                 }
4138 |             });
4139 |             const allTrainsData = await Promise.all(trainPromises);
4140 |
4141 |             if (requestId !== routeDataRequestId) return;
4142 |
4143 |             const cards = [];
4144 |             const totals = {
4145 |                 travelMinutes: 0,
4146 |                 turnaroundMinutes: 0,
4147 |                 trainCount: 0,
4148 |                 totalMileageKm: 0
4149 |             };
4150 |
4151 |             for (let i = 0; i < inputs.length; i++) {
4152 |                 const trainNumber = inputs[i];
4153 |                 const train = allTrainsData[i];
4154 |
4155 |                 if (train) {
4156 |                     totals.trainCount += 1;
4157 |                     if (train.duration !== null && !isNaN(train.duration)) {
4158 |                         totals.travelMinutes += train.duration;
4159 |                     }
4160 |
4161 |                     const firstTitle = train.stops[0]?.station?.title || '';
4162 |                     const lastTitle = train.stops[train.stops.length - 1]?.station?.title || '';
4163 |                     const mileageKm = calcRouteMileageKm(firstTitle, lastTitle);
4164 |                     if (mileageKm !== null) {
4165 |                         totals.totalMileageKm += mileageKm;
4166 |                     }
4167 |
4168 |                     let durationHtml = `В пути: ${formatMinutesToHoursAndMinutes(train.duration)}`;
4169 |                     let turnaroundHtml = '';
4170 |
4171 |                     if (i < inputs.length - 1) {
4172 |                         const nextTrain = allTrainsData[i + 1];
4173 |                         if (nextTrain) {
4174 |                             const arrCurrentMinutes = parseTimeToMinutes(train.arrival);
4175 |                             const depNextMinutes = parseTimeToMinutes(nextTrain.departure);
4176 |
4177 |                             if (arrCurrentMinutes !== null && depNextMinutes !== null) {
4178 |                                 let turnaroundMinutes = depNextMinutes - arrCurrentMinutes;
4179 |                                 if (turnaroundMinutes < 0) {
4180 |                                     turnaroundMinutes += 1440;
4181 |                                 }
4182 |                                 totals.turnaroundMinutes += turnaroundMinutes;
4183 |                                 turnaroundHtml = `Оборот: ${formatMinutesToHoursAndMinutes(turnaroundMinutes)}`;
4184 |
4185 |                                 const nextFirstTitle = nextTrain.stops[0]?.station?.title || '';
4186 |                                 const turnaroundKm = calcTurnaroundMileageKm(lastTitle, nextFirstTitle);
4187 |                                 if (turnaroundKm !== null) {
4188 |                                     totals.totalMileageKm += turnaroundKm;
4189 |
4190 |                                     if (turnaroundKm > 0) {
4191 |                                         turnaroundHtml += ` · ${formatMileage(turnaroundKm)}`;
4192 |                                     }
4193 |                                 }
4194 |                             }
4195 |                         }
4196 |                     }
4197 |                 }
4198 |             }

```

```

4195 |         }
4196 |         let currentDirection = 'to_moscow';
4197 |         if (firstTitle.toLowerCase().includes('москва')) {
4198 |             currentDirection = 'from_moscow';
4199 |         }
4200 |
4201 |         const mileageHtml = formatMileage(mileageKm);
4202 |         const cardDateLabel = formatTrainCardDate(train.scheduleDate, train.departure,
train.arrival);
4203 |
4204 |         let brakeHtml = '';
4205 |         if (sprDataCache) {

```

7. Нормализация и выбор шаблона смены

Файл: app/index.html; строки 6241–6425. Обработка дат и маркеров дней недели, классификация дневных, ночных и утренних маршрутов. ПОИСК ТОЧНОГО И РЕЗЕРВНОГО НОРМАТИВА.

```

6241 |     const SHIFT_WEEKDAY_MARKERS = ['пн', 'вт', 'ср', 'чт', 'пт', 'сб', 'вс', 'пн-чт', 'пн-пт', 'пн-пя'];
6242 |
6243 |     function normalizeWeekdayMarker(value) {
6244 |         return String(value || '').trim().toLowerCase().replace(/[-]/g, '-');
6245 |     }
6246 |
6247 |     function normalizeShiftFieldKey(key) {
6248 |         return String(key || '').toLowerCase().replace(/[/\s+\/g, '');
6249 |     }
6250 |
6251 |     function shiftPickField(raw, keys) {
6252 |         const normalizedMap = {};
6253 |         Object.entries(raw || {}).forEach(([k, v]) => {
6254 |             if (v == null || String(v).trim() === '') return;
6255 |             normalizedMap[normalizeShiftFieldKey(k)] = String(v).trim();
6256 |         });
6257 |         for (const key of keys) {
6258 |             const hit = normalizedMap[normalizeShiftFieldKey(key)];
6259 |             if (hit != null) return hit;
6260 |         }
6261 |         return '';
6262 |     }
6263 |
6264 |     function isShiftWeekdayMarker(value) {
6265 |         const v = normalizeWeekdayMarker(value);
6266 |         if (SHIFT_WEEKDAY_MARKERS.includes(v)) return true;
6267 |         return ['пн', 'вт', 'ср', 'чт', 'пт', 'сб', 'вс'].includes(v);
6268 |     }
6269 |
6270 |     function parseShiftDateToNormalized(dateStr) {
6271 |         const raw = String(dateStr || '').trim();
6272 |         if (!raw) return null;
6273 |
6274 |         const lower = normalizeWeekdayMarker(raw);
6275 |         if (isShiftWeekdayMarker(lower)) return lower;
6276 |
6277 |         const ymd = raw.match(/^(\\d{4})[.-/](\\d{1,2})[.-/](\\d{1,2})$/);
6278 |         if (ymd) {
6279 |             const [, y, m, d] = ymd;
6280 |             return `${String(d).padStart(2, '0')}-${String(m).padStart(2, '0')}-${String(y)}`;
6281 |         }
6282 |
6283 |         const dmy = raw.match(/^(\\d{1,2})[.-/](\\d{1,2})[.-/](\\d{4})$/);
6284 |         if (dmy) {
6285 |             const [, d, m, y] = dmy;
6286 |             return `${String(d).padStart(2, '0')}-${String(m).padStart(2, '0')}-${String(y)}`;
6287 |         }
6288 |
6289 |         return null;
6290 |     }
6291 |
6292 |     function normalizeShiftRecord(raw) {
6293 |         const route = shiftPickField(raw, ['route', 'маршрут', 'Route']);
6294 |         if (!route) return null;
6295 |
6296 |         const dateRaw = shiftPickField(raw, ['date', 'дата', 'Date']);
6297 |         const normalizedDate = parseShiftDateToNormalized(dateRaw) || dateRaw || 'пн-чт';
6298 |
6299 |         return {
6300 |             route: route,
6301 |             hours: shiftPickField(raw, ['hours', 'workHours', 'часы', 'Hours', 'рабочие часы']),
6302 |             startTime: shiftPickField(raw, ['startTime', 'время начала', 'Начало']),
6303 |             endTime: shiftPickField(raw, ['endTime', 'время окончания', 'конец', 'Конец']),
6304 |             lunch: shiftPickField(raw, ['lunch', 'обед', 'colI', 'Lunch']),
6305 |             trainNum: shiftPickField(raw, ['trainNum', 'номер поезда', 'trains', 'Trains']),
6306 |             nightHours: shiftPickField(raw, ['nightHours', 'ночные часы', 'ночные']),
6307 |             morningRoute: shiftPickField(raw, ['morningRoute', 'morningroute', 'утренний маршрут',
'утро']),
6308 |             date: normalizedDate
6309 |         };
6310 |     }
6311 |
6312 |     function normalizeShiftsArray(shifts) {
6313 |         if (!Array.isArray(shifts)) return [];
6314 |         return shifts.map(normalizeShiftRecord).filter(Boolean);
6315 |     }
6316 |
6317 |     function normalizedDateToCalendarKey(dateStr) {
6318 |         if (!dateStr || isShiftWeekdayMarker(dateStr)) return null;
6319 |         const parts = String(dateStr).split('-');

```

```

6320 |         if (parts.length !== 3) return null;
6321 |         const [d, m, y] = parts;
6322 |         return `${d}.${m}.${y}`;
6323 |     }

6324 |
6325 |     function calendarKeyToIso(dateKey) {
6326 |         const parts = String(dateKey || '').split('.');
6327 |         if (parts.length !== 3) return null;
6328 |         const [d, m, y] = parts;
6329 |         return `${y}-${m.padStart(2, '0')}-${d.padStart(2, '0')}`;
6330 |     }
6331 |
6332 |     function normalizedDateToIso(dateStr) {
6333 |         if (!dateStr || isShiftWeekdayMarker(dateStr)) return null;
6334 |         const parts = String(dateStr).split('-');
6335 |         if (parts.length !== 3) return null;
6336 |         const [d, m, y] = parts;
6337 |         return `${y}-${m.padStart(2, '0')}-${d}`;
6338 |     }
6339 |
6340 |     function getWeekdayMarkerForCalendarDate(dateKey) {
6341 |         const parts = String(dateKey || '').split('.');
6342 |         if (parts.length !== 3) return null;
6343 |
6344 |         const day = parseInt(parts[0], 10);
6345 |         const month = parseInt(parts[1], 10) - 1;
6346 |         const year = parseInt(parts[2], 10);
6347 |         const dow = new Date(year, month, day).getDay();
6348 |
6349 |         const singleMap = { 0: 'bc', 1: 'nh', 2: 'bt', 3: 'cp', 4: 'ct', 5: 'nt', 6: 'c6' };
6350 |         const single = singleMap[dow];
6351 |         let group = 'bc';
6352 |         if (dow >= 1 && dow <= 4) group = 'nh-ct';
6353 |         else if (dow === 5) group = 'nt';
6354 |         else if (dow === 6) group = 'c6';
6355 |
6356 |         return { single, group };
6357 |     }
6358 |
6359 |     function shiftDateMatchesWeekday(shiftDate, weekdayInfo) {
6360 |         if (!weekdayInfo) return false;
6361 |         const marker = normalizeWeekdayMarker(shiftDate);
6362 |         if (!isShiftWeekdayMarker(marker)) return false;
6363 |         if (marker === weekdayInfo.single || marker === weekdayInfo.group) return true;
6364 |         if (marker === 'nh-ct' && ['nh', 'bt', 'cp', 'ct'].includes(weekdayInfo.single)) return true;
6365 |         if ((marker === 'nh-nt' || marker === 'nh-ny') && ['nh', 'bt', 'cp', 'ct'].
'nt'].includes(weekdayInfo.single)) return true;
6366 |         return false;
6367 |     }
6368 |
6369 |     function findShiftTemplate(templates, routeNumber, dateKey) {
6370 |         const route = String(routeNumber || '').trim();
6371 |         if (!route || !dateKey) return null;
6372 |
6373 |         const pool = hasLocalShiftTemplates()
6374 |             ? getActiveShiftTemplates(dateKey)
6375 |             : (Array.isArray(templates) && templates.length ? templates : depotShiftTemplates);
6376 |         if (!Array.isArray(pool) || !pool.length) return null;
6377 |
6378 |         const cleanTarget = route.toUpperCase().replace(/[НУДНУ]/g, '').trim();
6379 |
6380 |         const isoDate = calendarKeyToIso(dateKey);
6381 |         const weekdayInfo = getWeekdayMarkerForCalendarDate(dateKey);
6382 |
6383 |         const getRouteKind = (routeStr) => {
6384 |             const r = String(routeStr || '').trim().toUpperCase();
6385 |             if (/^[HN]/.test(r)) return 'night';
6386 |             if (/^[YU]/.test(r) || /Y$/.test(r)) return 'morning';
6387 |             if (/^[DQ]/.test(r)) return 'day';
6388 |             return 'other';
6389 |         };
6390 |
6391 |         const targetKind = getRouteKind(route);
6392 |
6393 |         const isRouteMatch = (tplRoute) => {
6394 |             const rStr = String(tplRoute || '').trim();
6395 |             if (rStr === route) return true;
6396 |
6397 |             const tplKind = getRouteKind(rStr);
6398 |             if (targetKind !== 'other' && tplKind !== 'other' && targetKind !== tplKind) {
6399 |                 return false;
6400 |             }
6401 |
6402 |             const cleanTpl = rStr.toUpperCase().replace(/[НУДНУ]/g, '').trim();
6403 |             return cleanTpl === cleanTarget && cleanTarget !== '';
6404 |         };
6405 |
6406 |         for (let i = pool.length - 1; i >= 0; i--) {
6407 |             const row = pool[i];
6408 |             if (!isRouteMatch(row.route || row.Route)) continue;
6409 |             const rowIso = normalizedDateToIso(row.date);
6410 |             if (rowIso && isoDate && rowIso === isoDate) return { ...row };
6411 |         }
6412 |
6413 |         for (let i = pool.length - 1; i >= 0; i--) {
6414 |             const row = pool[i];
6415 |             if (!isRouteMatch(row.route || row.Route)) continue;
6416 |             if (shiftDateMatchesWeekday(row.date, weekdayInfo)) return { ...row };
6417 |         }

```

```

6418 |
6419 |         for (let i = pool.length - 1; i >= 0; i--) {
6420 |             const row = pool[i];
6421 |             if (isRouteMatch(row.route || row.Route)) return { ...row };
6422 |         }
6423 |
6424 |         return null;
6425 |     }

```

8. Рендер календаря и переход к маршруту

Файл: app/index.html; строки 9169–9295. Построение месячной сетки смен и передача поездов выбранной смены в модуль маршрутов.

```

9169 |     function calendarRenderCalendar() {
9170 |         const grid = document.getElementById('calendarGrid');
9171 |         if (!grid) return;
9172 |
9173 |         grid.innerHTML = CALENDAR_WEEKDAY_LABELS
9174 |             .map(label => `<div class="weekday">${label}</div>`)
9175 |             .join('');
9176 |
9177 |         const realToday = new Date();
9178 |
9179 |         const year = calendarCurrentDate.getFullYear();
9180 |         const month = calendarCurrentDate.getMonth();
9181 |
9182 |         document.getElementById('monthTitle').innerText = `${CALENDAR_MONTHS_RU[month]} ${year}`;
9183 |
9184 |         const firstDay = new Date(year, month, 1);
9185 |         let startDay = firstDay.getDay() - 1;
9186 |         if (startDay === -1) startDay = 6;
9187 |
9188 |         const totalDays = new Date(year, month + 1, 0).getDate();
9189 |
9190 |         for (let i = 0; i < startDay; i++) {
9191 |             const div = document.createElement('div');
9192 |             div.className = 'day other-month';
9193 |             grid.appendChild(div);
9194 |         }
9195 |
9196 |         loadGlobalShiftTemplates();
9197 |         if (syncMaterializedMorningShifts()) {
9198 |             persistUserShifts();
9199 |         }
9200 |
9201 |         for (let d = 1; d <= totalDays; d++) {
9202 |             const div = document.createElement('div');
9203 |             div.className = 'day';
9204 |
9205 |             if (year === realToday.getFullYear() && month === realToday.getMonth() && d ===
realToday.getDate()) {
9206 |                 div.classList.add('today');
9207 |             }
9208 |
9209 |             const dStr = String(d).padStart(2, '0');
9210 |
9211 |             const mStr = String(month + 1).padStart(2, '0');
9212 |             const dateKey = `${dStr}.${mStr}.${year}`;
9213 |
9214 |             const shifts = resolveShiftsForCalendarDay(dateKey);
9215 |
9216 |             div.dataset.dateKey = dateKey;
9217 |             div.onclick = () => calendarSelectDay(dateKey, d, month, year, div);
9218 |
9219 |             if (shifts.length === 0) {
9220 |                 div.innerHTML = `<span class="day-number">${d}</span>`;
9221 |             } else {
9222 |                 div.classList.add('day--has-shift');
9223 |                 if (shifts.length >= 2) div.classList.add('day--multi');
9224 |                 div.innerHTML = renderDayCellContent(d, shifts, dateKey);
9225 |             }
9226 |             grid.appendChild(div);
9227 |         }
9228 |
9229 |         if (calendarSelectedDateKey) {
9230 |             const parts = calendarSelectedDateKey.split('.');
9231 |             if (parts.length === 3) {
9232 |                 const selDay = parseInt(parts[0], 10);
9233 |                 const selMonth = parseInt(parts[1], 10) - 1;
9234 |                 const selYear = parseInt(parts[2], 10);
9235 |                 if (selYear === year && selMonth === month) {
9236 |                     const selDiv =
grid.querySelector(`.day[data-date-key="${calendarSelectedDateKey}"]`);
9237 |                     if (selDiv) selDiv.classList.add('selected');
9238 |                     renderShiftConstructor(calendarSelectedDateKey, selDay, selMonth, selYear);
9239 |                 }
9240 |             }
9241 |         }
9242 |
9243 |         function calendarChangeMonth(direction) {
9244 |             calendarCurrentDate.setDate(1);
9245 |             calendarCurrentDate.setMonth(calendarCurrentDate.getMonth() + direction);
9246 |             calendarRenderCalendar();
9247 |         }
9248 |
9249 |         function calendarFormatMonthGenitive(monthIndex) {
9250 |             const monthsGenitive = ['января', 'февраля', 'марта', 'апреля', 'мая', 'июня', 'июля', 'августа',

```

```

9251 | 'сентября', 'октября', 'ноября', 'декабря'];
9252 |     return monthsGenitive[monthIndex];
9253 | }
9254 |
9255 | function calendarSelectDay(dateKey, day, month, year, target) {
9256 |     document.querySelectorAll('#screen-calendar .day').forEach(d => d.classList.remove('selected'));
9257 |     if (target) target.classList.add('selected');
9258 |     calendarSelectedDateKey = dateKey;
9259 |     applyNormativeForDateKey(dateKey);
9260 |     renderShiftConstructor(dateKey, day, month, year);
9261 | }
9262 |
9263 | function navigateToRouteWithTrains(trainsText, dateKey, isNight = false, presetTrainNumbers = null,
9264 |     trainPlan = null) {
9265 |     let trainNumbers = Array.isArray(presetTrainNumbers) && presetTrainNumbers.length
9266 |         ? presetTrainNumbers.slice()
9267 |         : extractTrainNumbers(trainsText);
9268 |
9269 |     if (!trainNumbers.length) {
9270 |         if (typeof showToast === 'function') {
9271 |             showToast('Не удалось найти номера поездов');
9272 |         } else {
9273 |             alert('Не удалось найти номера поездов');
9274 |         }
9275 |     }
9276 |     return;
9277 | }
9278 |
9279 | resetSchedulePushState();
9280 |
9281 | switchMainScreen('schedule');
9282 |
9283 | const routeTabBtn = document.querySelector('.tab-btn[data-tab="tab-route"]');
9284 | if (routeTabBtn) {
9285 |     switchTab('tab-route', routeTabBtn);
9286 | }
9287 |
9288 | const iso = calendarKeyToIso(dateKey);
9289 | if (iso) {
9290 |     setSelectedDateIso(iso);
9291 | }
9292 |
9293 | renderTrainInputs(trainNumbers):
9294 |
9295 |     userStorageSet('trainNumbers', trainNumbers);
9296 |
9297 |     if (Array.isArray(trainPlan) && trainPlan.length) {
9298 |         userStorageSet('shiftTrainPlan', { items: trainPlan, dateKey });
9299 |     } else if (iso) {

```

9. Service Worker: автономная оболочка и уведомления

Файл: app/sw.js; строки 1–232. Версионированный прескейш, стратегии network-first/cache-first, очистка старых кешей и системные уведомления.

```

1 | // Версия релиза приложения – менять при каждом выкладке (сейчас 2.4.6.2 STABLE, deploy-ready)
2 | const CACHE_VERSION = 'da_v2_4_6_2';
3 | const NOTIFICATION_ICON = './assets/app-icon.png';
4 | const CACHE_NAME = `digital_assistant_${CACHE_VERSION}`;
5 |
6 | const PRECACHE_ASSETS = [
7 |     './',
8 |     './index.html',
9 |     './manifest.json',
10 |     './styles/tokens.css',
11 |     './styles/base.css',
12 |     './styles/routes.css',
13 |     './styles/schedule.css',
14 |     './styles/calendar.css',
15 |     './styles/profile.css',
16 |     './styles/responsive.css',
17 |     './styles/auth.css',
18 |     './styles/community.css',
19 |     './spr.json',
20 |     './trains-local.json',
21 |     './data/trains-uids.json',
22 |     './data/shift-templates.json',
23 |     './data/calendar-local-routes.json',
24 |     './data/release-notes.json',
25 |     './js/app-config.js',
26 |     './assets/brand-logo.png',
27 |     './assets/app-icon.png',
28 |     './nojekyll'
29 | ];
30 |
31 | function isPrecachePath(pathname) {
32 |     return PRECACHE_ASSETS.some((asset) => {
33 |         const cleanAsset = asset.replace(/^\.\/\./, '');
34 |         return pathname.endsWith(cleanAsset) || (cleanAsset === 'index.html' && pathname.endsWith('/'));
35 |     });
36 | }
37 |
38 | async function precacheAssets(cache) {
39 |     await Promise.allSettled(
40 |         PRECACHE_ASSETS.map((url) =>
41 |             cache.add(url).catch((err) => {
42 |                 console.warn(`[sw] пропуск прекеша ${url}: ${err}`);
43 |             })
44 |         )
45 |     );

```

```

46 | }
47 |
48 | async function matchNavigationFallback(cache) {
49 |   return cache.match('./index.html')
50 |     || cache.match('index.html')
51 |     || cache.match('./')
52 |     || cache.match('/');
53 | }
54 |
55 | async function matchCachedRequest(cache, request) {
56 |   const direct = await cache.match(request);
57 |   if (direct) return direct;
58 |
59 |   const url = new URL(request.url);
60 |   const path = url.pathname;
61 |   const fileName = path.split('/').pop() || '';
62 |
63 |   if (fileName) {
64 |     const byName = await cache.match(`.${${fileName}}`) || await cache.match(fileName);
65 |     if (byName) return byName;
66 |   }
67 |
68 |   if (request.mode === 'navigate' || path.endsWith('/')) {
69 |     return matchNavigationFallback(cache);
70 |   }
71 |
72 |   return null;
73 | }
74 |
75 | function revalidateInBackground(cache, request) {
76 |   fetch(request)
77 |     .then((response) => {
78 |       if (response && response.ok) {
79 |         cache.put(request, response.clone());
80 |       }
81 |     })
82 |     .catch(() => {});
83 | }
84 |
85 | function isAppShellRequest(pathname, mode) {
86 |   if (mode === 'navigate') return true;
87 |   return pathname.endsWith('/index.html') || pathname.endsWith('index.html');
88 | }
89 |
90 | function isReleaseNotesRequest(pathname) {
91 |   return pathname.endsWith('/data/release-notes.json') || pathname.endsWith('release-notes.json');
92 | }
93 |
94 | async function networkFirst(request) {
95 |   const cache = await caches.open(CACHE_NAME);
96 |
97 |   try {
98 |     const response = await fetch(request);
99 |     if (response && response.ok) {
100 |       cache.put(request, response.clone());
101 |     }
102 |     return response;
103 |   } catch (err) {
104 |     const cached = await matchCachedRequest(cache, request);
105 |     if (cached) return cached;
106 |     throw err;
107 |   }
108 | }
109 |
110 | async function cacheFirst(request) {
111 |   const cache = await caches.open(CACHE_NAME);
112 |   const cached = await matchCachedRequest(cache, request);
113 |
114 |   if (cached) {
115 |     revalidateInBackground(cache, request);
116 |     return cached;
117 |   }
118 |
119 |   try {
120 |     const response = await fetch(request);
121 |     if (response && response.ok) {
122 |       cache.put(request, response.clone());
123 |     }
124 |     return response;
125 |   } catch (err) {
126 |     const fallback = await matchCachedRequest(cache, request);
127 |
128 |     if (fallback) return fallback;
129 |     throw err;
130 |   }
131 | }
132 | self.addEventListener('install', (event) => {
133 |   event.waitUntil(
134 |     caches.open(CACHE_NAME)
135 |       .then((cache) => precacheAssets(cache))
136 |       .then(() => self.skipWaiting())
137 |       .catch((err) => console.warn('[sw] прекэш отклонён из-за отсутствия файлов:', err))
138 |   );
139 | });
140 |
141 | self.addEventListener('activate', (event) => {
142 |   event.waitUntil(
143 |     caches.keys()
144 |       .then((keys) => Promise.all(
145 |         keys.filter((key) => key !== CACHE_NAME).map((key) => caches.delete(key))

```

```

146 |         ))
147 |         .then(() => self.clients.claim())
148 |     );
149 | });
150 |
151 | async function showScheduleSystemNotification(payload = {}) {
152 |     const title = String(payload.title || 'Цифровой помощник').trim();
153 |     const kicker = String(payload.kicker || '').trim();
154 |     const bodyText = String(payload.body || '').trim();
155 |     const body = [kicker, bodyText].filter(Boolean).join(' - ') || title;
156 |     const tag = String(payload.tag || payload.key || 'schedule-alert').trim();
157 |
158 |     await self.registration.showNotification(title, {
159 |         body,
160 |         tag,
161 |         renotify: true,
162 |         icon: NOTIFICATION_ICON,
163 |         badge: NOTIFICATION_ICON,
164 |         vibrate: [140, 70, 140],
165 |         requireInteraction: true,
166 |         silent: false,
167 |         data: {
168 |             key: tag,
169 |             url: './',
170 |             screen: 'schedule'
171 |         }
172 |     });
173 | }
174 |
175 | self.addEventListener('message', (event) => {
176 |     const data = event.data;
177 |     if (!data || !data.type) return;
178 |
179 |     if (data.type === 'SKIP_WAITING') {
180 |         self.skipWaiting();
181 |         return;
182 |     }
183 |
184 |     if (data.type === 'SHOW_SCHEDULE_NOTIFICATION') {
185 |         event.waitUntil(
186 |             showScheduleSystemNotification(data.payload || {})
187 |                 .catch(err => console.warn('[sw] schedule notification failed:', err))
188 |         );
189 |     }
190 | });
191 |
192 | self.addEventListener('notificationclick', (event) => {
193 |     event.notification.close();
194 |     const targetUrl = event.notification.data?.url || './';
195 |
196 |     event.waitUntil(
197 |         self.clients.matchAll({ type: 'window', includeUncontrolled: true })
198 |             .then((clientList) => {
199 |                 for (const client of clientList) {
200 |                     if ('focus' in client) {
201 |                         return client.focus();
202 |                     }
203 |                 }
204 |                 if (self.clients.openWindow) {
205 |                     return self.clients.openWindow(targetUrl);
206 |                 }
207 |                 return null;
208 |             })
209 |     );
210 | });
211 |
212 | self.addEventListener('fetch', (event) => {
213 |     const { request } = event;
214 |     if (request.method !== 'GET') return;
215 |
216 |     const url = new URL(request.url);
217 |     if (url.origin !== self.location.origin) return;
218 |
219 |     if (isReleaseNotesRequest(url.pathname)) {
220 |         event.respondWith(networkFirst(request));
221 |         return;
222 |     }
223 |
224 |     if (isAppShellRequest(url.pathname, request.mode)) {
225 |         event.respondWith(networkFirst(request));
226 |         return;
227 |     }
228 |
229 |     if (isPrecachePath(url.pathname)) {
230 |         event.respondWith(cacheFirst(request));
231 |     }
232 | });

```


11. Контрольная ведомость

Хеши позволяют сопоставить листинг с файлами, использованными при формировании настоящего документа.

Файл	SHA-256
app/index.html	2dd4dfac2b7fdc81bc14b7d6621e2be55b26f9404c80aef4974d8dfdb1ea223f
app/sw.js	ab5f65f2cd86a7d65b861bde0926e40f8ac6b7608061fc870817dfeeeb11b36

12. Примечание о подаче

Настоящий файл представляет комплект идентифицирующих материалов и не заменяет заявление на государственную регистрацию. Текст реферата может дополнительно переноситься в соответствующее поле заявления. Требование к PDF/A и разделение реферата и листинга следует проверить для выбранного способа подачи в ФИПС.